A Provenance-Aware Design for Aggregating Competitive-Programming Profiles

Sajal Goyal, Shri Krishna Chandra Vaishnav, Yug Mittal, Ravi Shah, and Suryadev Singh

Arya College of Engineering & I.T., Jaipur, India

Advanced Java Laboratory / Project Based Learning, 2026

Abstract

Competitive-programming learners commonly maintain accounts on multiple online judges, while each platform exposes a different subset of statistics and history. Consolidating such records can support convenient review, but naïve aggregation risks treating unlike measures as equivalent, turning missing data into false zeroes, or presenting unverified platform data as authoritative. This paper examines the design of KodBTW, a Java-based web application that integrates coding-platform accounts with dashboards, persisted snapshots, analytics, leaderboards, and public profiles. We use a structured narrative review of 30 scholarly works on learning analytics dashboards, programming-process analytics, and online judges, followed by a source-code and Flyway-schema audit of the checked-out artifact. The implementation has live adapters for LeetCode and Codeforces, while CodeChef, GeeksforGeeks, and HackerRank accounts remain connectable without live statistics synchronization. Platform observations are stored with source tags and dates; unsupported fields remain nullable, and historical views are based on recorded snapshots. The application also applies JWT authentication and user ownership checks to protected API operations. These findings demonstrate a concrete provenance-aware engineering design, not educational effectiveness. Automated tests and a running application could not be verified in the available environment because Java/JDK and npm were unavailable. We conclude with a validation plan for data correctness, user experience, and future learning-outcome evaluation.

Index Terms— learning analytics, competitive programming, online judge, data provenance, software artifact, dashboard.

I. INTRODUCTION

Online judges and coding-practice platforms provide learners with separate profiles, ratings, submission statistics, and activity histories. Prior research has studied both dashboards that summarize learner data and analytics derived from programming traces [2]-[15], [16]-[23]. Yet a multi-platform profile view introduces a specific engineering problem: the source systems do not expose the same fields, update schedules, or definitions. A cross-platform total may therefore mix distinct constructs or imply more certainty than the source supports.

This paper presents KodBTW, a web application developed for an Advanced Java Laboratory group project. The system is designed to collect selected public profile statistics from supported coding platforms, preserve dated observations, and present account-scoped dashboard and leaderboard views. The research contribution is a descriptive artifact analysis of how the implementation handles heterogeneous metrics, unavailable values, source status, and persisted history. It does not claim that dashboards improve learning, that the project has been deployed, or that all platform integrations work.

The paper is guided by three research questions:

• RQ1: How do learning-analytics and programming-education studies collect, normalize, visualize, and evaluate learner-progress data?

• RQ2: Which provenance and missing-data constraints are visible in KodBTW's adapter, persistence, and analytics design?

• RQ3: Which proposed capabilities remain outside the implemented artifact, and what evidence would be needed to evaluate them?

The scope is bounded to the repository snapshot identified in this report. The project proposal also describes mentor dashboards, campus skill-gap analysis, student verification, inactivity detection, workshop recommendations, and AI suggestions. These are treated as future proposals unless present in source code.

II. RELATED WORK

A. Learning analytics dashboards

Learning analytics dashboards translate digital traces into summaries intended for learners, instructors, or administrators. Research has examined dashboard applications, stakeholder views, data sources, visualization, and design opportunities [2]-[7]. Systematic reviews repeatedly note that many implementations are exploratory and that the connection between dashboard features and learning theory or measurable educational outcomes can be weak [4]-[11], [14]. Students' expectations research reports interest in planning, self-assessment, and personalized analyses, alongside concerns about privacy and learner autonomy [8].

Dashboard evaluation must distinguish access, use, perceived utility, behavior, and learning outcomes. Hellings and Haelermans studied dashboard access in a Java programming course with 556 first-year students. They report effects on some online performance measures but not on final exam performance or course passing, with subgroup differences [12]. This result motivates a strict boundary in KodBTW documentation: an implemented dashboard is a software feature, not evidence of improved learning.

B. Programming-process and online-judge analytics

Programming-education analytics draws on data unavailable in ordinary profile summaries, including code snapshots, IDE/compiler events, and submission histories [16]-[20]. Online-judge systems have long been studied as automatic assessment tools [23]. Recent work analyzes problem order and difficulty [21], competitive-programming instruction [22], programming behavior and prediction [26], and integrated programming platforms that collect learner behavior for feedback [28]. Reviews of programming analytics emphasize the need for replication, stronger context description, and careful interpretation across courses and datasets [16], [27].

These studies are relevant but not interchangeable with multi-platform profile data. A contest rating, a profile's solved total, an IDE compiler event, and a course exercise grade represent different observations. Prior evidence does not establish a universal scale across them. KodBTW therefore treats each source as a platform-scoped observation and does not claim topic-level mastery or a common cross-platform ability score.

C. Research gap

Prior research documents both dashboards and programming analytics, but the two bodies do not provide a common policy for combining public profile metrics from heterogeneous coding platforms. The actionable gap for this artifact is an implementation-level one: preserve provenance and observation time, distinguish unavailable values from zero, prevent unsupported integrations from being advertised as live, and avoid inventing historical values. This is a bounded engineering gap inferred from the reviewed literature and code audit; it is not a claim that no similar product exists.

D. Comparative synthesis and transfer limits

The 30-record register falls into overlapping evidence groups rather than one directly comparable set of experiments. Dashboard reviews and design studies [4]-[15] examine who receives analytics, what data are displayed, and whether interface goals connect to learning actions. Programming analytics and online-judge studies [16]-[23], [26]-[28] use code events, submission traces, exercise outcomes, and contest-related data. Feedback and introductory-programming reviews [24], [25], [29], [30] describe instructional tools whose primary purpose is to change how learners practice or receive guidance. The groups answer different questions, so a dashboard design result cannot be treated as a finding about coding-platform integration, and a submission-analysis result cannot be treated as evidence for public-profile metrics.

For learner-facing dashboards, the recurring design concern is interpretability: a visualization should help its intended user connect evidence with an action. A dashboard can display many measures and still leave the learner unsure what the measure means, how recent it is, or what to do next. The literature also describes privacy and agency concerns when analytics are designed for instructors or institutions rather than learners. This supports a user-centered interpretation of KodBTW's individual profile and dashboard routes. It does not establish the efficacy of those routes, and it provides no evidence for the proposal's future campus-monitoring functions.

For evaluation, the reviewed studies distinguish user perception, dashboard use, changes in practice, course activity, and academic outcomes. These are not interchangeable dependent variables. The randomized Java-course study [12] is a useful caution because observed differences in intermediate online performance did not translate to differences in final exam performance or course passing. Such evidence is not directly transferable to an extracurricular competitive-programming profile aggregator, but it demonstrates why an implemented dashboard cannot itself serve as an educational outcome. KodBTW has no participant data or intervention results; its current contribution is a system artifact whose behavior must be tested before any outcome study.

Programming-process research contributes richer temporal signals than the current application collects. Compilation events, code snapshots, task sequences, submissions, and feedback can reveal attempts, errors, and strategy changes [16]-[20], [24]-[26]. Public platform profile summaries are more compact and convenient, but they omit much of that process. A rating or solved-problem count is the result of platform-specific rules and a collection history; it cannot reveal what concepts a learner understands, the difficulty of each task, or whether a learner arrived at an answer independently. The project therefore avoids claiming DSA topic diagnosis from current account statistics.

Online-judge and competitive-programming research is closest in domain but still differs in data and setting. Course-based studies often know the assignment, rubric, task difficulty, and student cohort; a profile aggregator sees only selected public fields and may not know the context that produced them. Problem order

and difficulty can affect observed outcomes [21], while contest participation studies may depend on event-specific populations [22]. These results justify treating metric definitions and context as part of interpretation; they do not provide a common conversion formula for the application's supported sources.

The resulting evidence-to-design mapping is deliberately modest:

Evidence theme	Reported concern in prior work	KodBTW design implication	Current evidence level
Dashboard intent	A display should match the learner's goal and support interpretation	Keep account-level profile and progress views distinct from institutional monitoring	Routes and service scope inspected; usability not measured
Outcome evaluation	Use, engagement, and learning outcomes are different constructs	Do not claim effectiveness from feature presence or profile counts	No intervention or participant outcomes collected
Programming process	Course/IDE traces expose process that profile summaries omit	Avoid topic mastery or strategy claims from aggregate public fields	Current schema/adapters inspected
Data provenance	Source, time, and metric definitions matter to comparison	Retain source labels and dated snapshots; disclose unsupported fields	Persistence and adapter code inspected; runtime not executed
Privacy and agency	Learner-facing and staff-facing analytics have different access implications	Keep proposed campus/mentor views outside the current system description	Current route/security configuration inspected

This comparison describes how the reviewed literature informs requirements and interpretation. It is not a meta-analysis: study populations, metrics, research designs, and access levels are heterogeneous. Some references were available only as abstracts or publisher records, and the record-level access field in the CSV should be consulted before drawing paper-specific conclusions. The synthesis uses full-text claims only where the underlying source was available and otherwise limits itself to bibliographic/abstract-level relevance.

III. METHOD

A. Literature review

We used a structured narrative review of 30 scholarly records published across learning analytics dashboards, programming education, online judges, and programming feedback. Search terms included learning analytics dashboard, student-facing dashboard, programming education analytics, programming behavior data, programming online judge, and competitive programming. Publisher pages and scholarly repositories were checked for stable bibliographic identity, venue, DOI or persistent record, abstract, and source availability. The reference register records whether an open full text or only metadata/abstract was accessible. Records with abstract-only access are not presented as full-text evaluations.

This review is not a systematic review: it did not execute exhaustive queries against a fixed set of indexed databases, independent screening, or complete extraction of all full texts. The standalone review provides the search limits and record-level access state. Its purpose is to ground the artifact questions and avoid unsupported claims.

B. Artifact audit

The artifact audit inspected the checked-out source tree, REST controllers, platform adapters, JPA entities, Flyway migrations V1-V7, frontend routes, repository test sources, and project setup notes. The primary audit date was 10 October 2026. The repository state examined was commit 1d45df1; the checkout was shallow, and a remote history fetch failed. The audit therefore does not infer complete contribution history from Git.

We mapped implementation evidence to four design properties: (1) source attribution, (2) date-preserving persistence, (3) explicit handling of unsupported/missing metrics, and (4) owner-scoped security. We also compared the proposal's suggested capabilities with actual routes and classes. No user study, benchmark, or production deployment was performed.

C. Verification constraints

The backend test suite could not start because the environment reported JAVA_HOME not found. Frontend dependency installation could not start because npm was unavailable. MySQL credentials/service were not provided. Consequently, source inspection is reported as inspection, test classes are reported as present, and no test-pass/build/deployment result is claimed.

IV. SYSTEM DESIGN

A. Architecture

KodBTW has a React 18/TypeScript/Vite frontend and a Java 21/Spring Boot REST backend. The API calls application services, security filters, adapter clients, and Spring Data JPA repositories. MySQL 8 is managed by ordered Flyway migrations. JWTs are sent using an Authorization bearer header. The deployment diagram in Fig. 1 describes the intended topology, not a verified deployed environment.

Fig. 1. Intended deployment topology of KodBTW, based on repository configuration. The static-hosting and database nodes have not been verified as production services.

B. Platform source model

Platform enumerates LeetCode, CodeChef, Codeforces, GeeksforGeeks, and HackerRank. The method hasLiveStatsSource() returns true only for LeetCode and Codeforces. The Codeforces adapter calls the public user.info endpoint and maps current rating and maximum rating while returning unsupported metrics as null. The LeetCode adapter maps selected profile, submission, contest, and calendar fields from a publicly reachable but undocumented GraphQL query. Its endpoint contract and platform terms are not established by the application code. The remaining adapters explicitly throw a sync-unavailable exception; no website scraping or mock fallback is used as a live source.

These implementation distinctions matter for data interpretation. For example, Codeforces user information does not provide a total-solved count or contest history through the mapped endpoint. A missing solved metric is not equivalent to zero solved problems. CodeChef/GFG/HackerRank account records can be stored, but the current application cannot refresh their statistics.

C. Snapshot and analytics model

Flyway migrations V1-V7 define users, profiles, platform accounts, platform-stat snapshots, and a denormalized leaderboard cache. A snapshot is unique by (user_id, platform, snapshot_date), stores a source tag, and contains nullable metric columns. The cache is keyed by user and is rebuilt from stored statistics. Current implementation notes and code use source status to keep unavailable/mock data from being represented as verified live statistics. Historical analytics query persisted observations; the code does not synthesize historical points where observations are absent.

TABLE I. Domain data model summary

Table	Main purpose	Key constraint
users	Authentication identity	Unique email
profiles	User profile and public username	Unique user FK; unique username
platform_accounts	Connected platform handle and sync status	Unique user/platform pair
platform_stat_snapshots	Dated platform metrics with provenance	Unique user/platform/date; nullable metrics
leaderboard_user_cache	Rebuildable per-user aggregate projection	Shared user PK/FK

The isolated schema_initialization_check table from V1 is a migration utility, not a domain entity. The complete schema and cardinalities are in the ER diagram and database documentation.

D. Access control

E. Metric interpretation and data lineage

A useful analysis unit in this system is not an unqualified “student score” but a metric observation whose meaning depends on a platform, account, collection time, and adapter mapping. This framing follows directly from the implementation structure: adapters return a platform-specific statistics object, the synchronization service records a snapshot, and downstream dashboard/history services read stored observations. The model does not establish that values from distinct platforms can be summed or ranked on a single scale.

The schema's source and timestamp fields are necessary but not sufficient for reproducibility. A production-grade record would also benefit from an adapter/version identifier, upstream response timestamp, metric-definition version, and a stable source reference where terms permit. The current code audit does not establish all these fields as implemented, so they are recommendations rather than features. Their purpose would be to answer which mapping produced an observation and whether two observations were collected under equivalent definitions.

Missingness also has more than one cause. A metric may be structurally unsupported by an endpoint, temporarily unavailable due to upstream failure, absent from the provider response, or not yet collected. Nullable fields preserve the difference between zero and unknown at the data layer, while sync status can convey an operation outcome. A consumer can still erase that distinction by rendering null as zero or aggregating stale values. Accordingly, future UI validation should inspect both persisted state and final rendered labels.

This yields four practical interpretation rules for the present artifact: (1) compare ratings only within platform-specific contexts unless a justified normalization exists; (2) present unavailable data as unavailable;

(3) describe history as observations captured by KodBTW rather than a complete platform archive; and (4) identify which platforms currently support refresh. These rules describe intended safe interpretation; no usability study has yet shown that users understand the interface this way.

F. Security and privacy boundary

The application security model is user-centric. The inspected configuration is stateless and JWT-based, and protected account operations use authenticated identity and ownership checks. Public registration, login, health, and public-profile paths are treated separately. A manual leaderboard rebuild route is restricted to an administrator role. These are code-level findings, not a penetration-test result.

A public profile makes selected profile data intentionally discoverable. The distinction between public profile information and private account credentials must remain explicit in response DTOs and tests. The project proposal's campus-wide views would change the privacy boundary substantially: they could expose cohort participation, activity, or perceived inactivity to staff. Those capabilities are not present in the routes or schema audited here. If introduced, they would require purpose limitation, access scopes, retention controls, student notice, and a documented institutional policy before technical implementation.

G. Reliability and adapter failure model

External data collection has at least three distinct failure domains: the platform endpoint, the adapter mapping, and persistence/cache update after a response. The current platform synchronization flow and status fields provide a basis for reporting operation outcomes, but the project has not been executed here to validate retries, timeout behavior, or partial failure recovery. The LeetCode adapter's undocumented upstream contract adds drift risk; a response shape change can cause a mapping failure even where the user's public page remains available.

A future reliability test suite should use authorized fixtures for expected response shapes, malformed and partial payloads, network timeouts, rate limiting, and persistence errors. The test oracle should distinguish a valid response with a zero value from missing data and a failed request. Live calls should be isolated from deterministic tests. Documentation should also identify endpoint stability and platform rules; a technically reachable endpoint is not evidence of permission for unrestricted collection.

V. RESULTS OF THE ARTIFACT AUDIT

Spring Security is stateless and uses a JWT filter. Public routes include registration, login, health, and public profile reads. Other routes require authentication. Platform-account operations are performed for the authenticated user and service methods enforce ownership. Manual leaderboard rebuild is restricted to an ADMIN authority. The system does not implement the proposal's distinct mentor or college-administrator workflows.

V. RESULTS OF THE ARTIFACT AUDIT

A. Implemented functions

The frontend defines routes for landing, login, registration, public profile, dashboard, analytics, contests, leaderboard, profile, and coding profiles. Backend controllers expose account/profile operations, current-user authentication, dashboard statistics, history and insights, activity/contest summaries, leaderboard reads, and public profiles. A scheduled leaderboard synchronization component exists in source. The audit does not establish that any server was running during inspection.

B. Provenance and missing-data observations

The source and schema support three relevant safeguards:

• Platform-scoped source tags: stored snapshots identify an origin such as LEETCODE_REAL or CODEFORCES_REAL.

• Nullable unsupported fields: adapter mappings leave metrics null when the endpoint does not provide them.

• Dated snapshots: history is based on observations saved by sync, with uniqueness constraints that preserve a daily observation model.

This design reduces the risk of false zeros and fabricated time-series data. It does not prove that every upstream value is correct, current, or authorized. LeetCode's undocumented query remains a stability and terms-review dependency. Codeforces' endpoint exposes only a limited subset of fields.

C. Feature boundary

TABLE II. Proposal-to-implementation comparison

Proposed capability	Current implementation evidence
Unified profiles and account links	Implemented for five platform enum values
Live statistics from every named platform	Only LeetCode and Codeforces have live adapters; the other three are explicitly unavailable
Dashboard and history	Implemented using stored data; no historical backfill is claimed
Leaderboard	Authenticated global/college reads and cache are present
Public developer profile	Public username route and DTO-backed service are present
Mentor and college-admin monitoring	Not found in the route/security model
Topic-level DSA skill-gap analysis	Not found in current schema or service routes
Inactivity detection and workshop recommendation	Not found in current implementation
AI-generated learning recommendations	Not found in current implementation
Profile ownership verification workflow	A verified field exists, but a verification workflow was not established

The proposal's broader campus-intelligence vision must therefore be presented as future scope. The project report's SRS and UML diagrams use the actual user roles and workflows.

D. Verification status

Test sources cover controllers, services, adapters, security/CORS, exception mapping, and migration behavior. They were not executed in this environment. Thus, the results are source-audit findings, not measured runtime outcomes. No application screenshots, benchmark, load test, or user evaluation is included as if executed.

E. Proposed validation protocol (not executed)

The audit exposes a validation sequence that separates correctness, reliability, user comprehension, and educational outcomes. The stages should not be collapsed into a single “works” claim.

Stage 1: deterministic mapping checks. For each supported adapter, keep consent-compatible or synthetic test fixtures that include complete data, partial data, no data, and malformed data. Assert exact field mappings, account identity, source tag, collection time, and preservation of nulls. This checks the adapter contract without depending on a live provider. Codeforces and LeetCode require separate expected mappings because their available fields differ.

Stage 2: persistence and ownership checks. Apply Flyway migrations V1-V7 to a clean MySQL 8 database and verify the domain relationships, uniqueness rules, and nullable columns. Exercise authenticated requests using two distinct users to confirm that one user cannot read, change, delete, or synchronize another user's account. Test public-profile responses separately to ensure they return intended public fields without exposing credentials. No result is reported for these checks because the configured environment lacked Java and MySQL.

Stage 3: interface comprehension and task usability. Give representative students tasks such as linking an account, interpreting a missing metric, reading a trend, and identifying whether a value is current. Observe task completion and ask participants to explain what the metric represents and when it was collected. A null-versus-zero comprehension item is particularly important because the database representation alone does not ensure correct interpretation. Record task success, time, errors, and user explanations; obtain informed consent and do not include personal data in screenshots or the paper.

Stage 4: learning-outcome evaluation. Only after the interface and data are validated should a study test whether using a dashboard changes a defined educational outcome. A protocol should pre-register a learning construct, intervention duration, comparison condition, measures, participant inclusion criteria, sample-size rationale, and analysis. Public rating changes alone should not be used as a proxy for learning without construct validation. Any campus-monitoring proposal also needs explicit governance and opt-in or institutional authorization.

This staged plan is a methodological implication of the artifact and literature, not evidence that any evaluation has occurred. It avoids treating a successful HTTP response, visually populated dashboard, user satisfaction, and improved learning as interchangeable outcomes.

VI. DISCUSSION

The artifact aligns with a recurring literature concern: analytics should expose the provenance and limits of the data used to form a display. By preserving source and date and representing absent fields as null, KodBTW makes some important limitations explicit at the data model boundary. Separate platform adapters localize differences in upstream schema. The explicit unavailable state prevents the application from treating connectability as evidence that a platform allows stable automated statistics collection.

These design choices are not sufficient for educational effectiveness. A profile dashboard that presents rating or solved counts can aid convenience, but the relationship between these indicators and a learner's mastery, strategy, or outcomes has not been validated. Studies in course settings show that some dashboard exposure can affect intermediate activity without changing final performance [12]. Programming-process studies often use richer, more contextual data than public profiles [16]-[21], [26]. KodBTW therefore cannot infer DSA topic strengths from its present data.

There is also a source-permission and stability limitation. Codeforces exposes a documented public API for the mapped user information. The LeetCode implementation uses an undocumented query; its availability is not a durable contract. A future release should obtain authorized/stable interface evidence, document field definitions and caching constraints, and prevent unsupported adapters from returning mock values as live output.

VII. THREATS TO VALIDITY AND LIMITATIONS

Construct validity: “coding progress” is represented by platform-specific public metrics and snapshots, not by a validated construct of learning or proficiency. Ratings, solved totals, contest participation, and activity are not interchangeable.

Internal validity: No experiment, participant sample, or intervention was run. The artifact audit cannot establish that the application changes user behavior or learning outcomes.

External validity: Source inspection covers one repository snapshot and the adapters/fields in that version. Upstream schemas and terms may change. Findings do not generalize to all platforms or institutions.

Reproducibility: The checkout was shallow, full history retrieval failed, and tests could not execute. Team contribution records and the 90-day portal logs are incomplete in the supplied evidence. The PBL portal export shows no submitted daily/weekly logs. These gaps are documented separately rather than reconstructed as facts.

Literature review: The 30-record set is a structured narrative review, not an exhaustive systematic review. Some records were accessible only through abstract/metadata, as recorded in the database. No claim of full-text evaluation is made for these items.

VIII. FUTURE WORK

Future technical validation should test adapter mappings against permitted fixtures, confirm schema migration behavior on MySQL 8, verify owner scoping and source filtering, and measure response reliability under upstream failure. A usability study can then measure task completion, comprehension of null/availability states, and interpretation of cross-platform comparisons. Any learning-impact study should define a theory-driven intervention, a comparison condition, validated outcome measures, consent/data governance, and adequate sample size. Topic-level skills, institution-wide dashboards, and recommendations require new data sources and requirements; they should not be inferred from the present profile data.

IX. CONCLUSION

KodBTW implements a multi-platform profile and analytics system with clear current boundaries. Its two live sources, source-tagged snapshots, nullable metrics, user-scoped APIs, and explicit unavailable adapters provide a traceable basis for present functionality. The code audit supports claims about design and implementation only. It does not support a claim that the application has passed its tests in the examined environment, is deployed, or improves student learning. Future work should validate data mappings, upstream permissions, usability, and educational outcomes as separate questions.

REFERENCES

[1] Z. Papamitsiou, A. A. Economides, ‘Learning Analytics and Educational Data Mining in Practice: A Systematic Literature Review of Empirical Evidence,’ Educational Technology & Society 17(4), 49-64, 2014. https://eric.ed.gov/?id=EJ1045537.

[2] K. Verbert, E. Duval, J. Klerkx, S. Govaerts, J. L. Santos, ‘Learning analytics dashboard applications,’ American Behavioral Scientist 57(10), 1500-1509, 2013. https://doi.org/10.1177/0002764213479363.

[3] K. Verbert, S. Govaerts, E. Duval, J. L. Santos, F. Van Assche, G. Parra, J. Klerkx, ‘Learning dashboards: an overview and future research opportunities,’ Personal and Ubiquitous Computing 18(6), 1499-1514, 2014. https://doi.org/10.1007/s00779-013-0751-2.

[4] B. Schwendimann et al., ‘Perceiving Learning at a Glance: A Systematic Literature Review of Learning Dashboard Research,’ IEEE Transactions on Learning Technologies 10(1), 30-41, 2017. https://doi.org/10.1109/TLT.2016.2599522.

[5] R. Bodily, K. Verbert, ‘Review of Research on Student-Facing Learning Analytics Dashboards and Educational Recommender Systems,’ IEEE Transactions on Learning Technologies 10(4), 405-418, 2017. https://doi.org/10.1109/TLT.2017.2740172.

[6] I. Jivet, M. Scheffel, M. Specht, H. Drachsler, ‘License to evaluate: preparing learning analytics dashboards for educational practice,’ Proceedings of LAK 2018, 31-40, 2018. https://doi.org/10.1145/3170358.3170421.

[7] R. G. Bodily, T. K. Ikahihifo, B. Mackley, C. R. Graham, ‘The design, development, and implementation of student-facing learning analytics dashboards,’ Journal of Computing in Higher Education 30(3), 572-598, 2018. https://doi.org/10.1007/s12528-018-9186-0.

[8] C. Schumacher, D. Ifenthaler, ‘Features students really expect from learning analytics,’ Computers in Human Behavior 78, 397-407, 2018. https://doi.org/10.1016/j.chb.2017.06.030.

[9] W. Matcha, N. A. Ahmad Uzir, D. Gaševi■, A. Pardo, ‘A Systematic Review of Empirical Studies on Learning Analytics Dashboards: A Self-Regulated Learning Perspective,’ IEEE Transactions on Learning Technologies 13(2), 226-245, 2020. https://doi.org/10.1109/TLT.2019.2916802.

[10] N. Valle, P. Antonenko, K. Dawson, A. C. Huggins-Manley, ‘Staying on target: A systematic literature review on learner-facing learning analytics dashboards,’ British Journal of Educational Technology 52, 1724-1748, 2021. https://doi.org/10.1111/bjet.13089.

[11] G. Ramaswami, T. Susnjak, A. Mathrani et al., ‘Use of Predictive Analytics within Learning Analytics Dashboards: A Review of Case Studies,’ Technology, Knowledge and Learning 28, 959-980, 2023. https://doi.org/10.1007/s10758-022-09613-x.

[12] J. Hellings, C. Haelermans, ‘The effect of providing learning analytics on student behaviour and performance in programming: a randomised controlled experiment,’ Higher Education 83, 1-18, 2022. https://doi.org/10.1007/s10734-020-00560-z.

[13] M. Zhu, A. R. Sari, M. M. Lee, ‘Trends and Issues in MOOC Learning Analytics Empirical Research: A Systematic Literature Review (2011-2021),’ Education and Information Technologies 27, 10135-10160, 2022. https://doi.org/10.1007/s10639-022-11031-6.

[14] L. Paulsen, E. Lindsay, ‘Learning analytics dashboards are increasingly becoming about learning and not just analytics - A systematic review,’ Education and Information Technologies 29, 14279-14308, 2024. https://doi.org/10.1007/s10639-023-12401-4.

[15] O. Viberg, M. Hatakka, O. Bälter, A. Mavroudi, ‘The current landscape of learning analytics in higher education,’ Computers in Human Behavior 89, 98-110, 2018. https://doi.org/10.1016/j.chb.2018.07.027.

[16] P. Ihantola et al., ‘Educational data mining and learning analytics in programming: Literature review and case studies,’ Proceedings of ITiCSE Working Group Reports, 41-63, 2016. https://doi.org/10.1145/2858796.2858798.

[17] P. Blikstein, ‘Using learning analytics to assess students' behavior in open-ended programming tasks,’ Proceedings of the 1st International Conference on Learning Analytics and Knowledge, 110-116, 2011. https://doi.org/10.1145/2090116.2090132.

[18] P. Blikstein, ‘Programming Pluralism: Using Learning Analytics to Detect Patterns in the Learning of Computer Programming,’ Journal of the Learning Sciences 23(4), 2014. https://doi.org/10.1080/10508406.2014.954750.

[19] A. Altadmri, N. C. C. Brown, ‘37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data,’ Proceedings of SIGCSE 2015, 522-527, 2015. https://doi.org/10.1145/2676723.2677258.

[20] S. López-Pernas, M. Saqr, O. Viberg, ‘Putting It All Together: Combining Learning Analytics Methods and Data Sources to Understand Students' Approaches to Learning Programming,’ Sustainability 13(9), 4825, 2021. https://doi.org/10.3390/su13094825.

[21] J. Wang, P. Lin, Z. Tang, S. Chen, ‘How problem difficulty and order influence programming education outcomes in online judge systems,’ Heliyon 9(11), e20947, 2023. https://doi.org/10.1016/j.heliyon.2023.e20947.

[22] K. K. F. Yuen, D. Y. W. Liu, H. V. Leong, ‘Competitive programming in computational thinking and problem solving education,’ Computer Applications in Engineering Education 31, 850-866, 2023. https://doi.org/10.1002/cae.22610.

[23] A. Kurnia, A. Lim, B. Cheang, ‘Online judge,’ Computers & Education 36(4), 299-315, 2001. https://doi.org/10.1016/S0360-1315(01)00018-5.

[24] H. Keuning, J. Jeuring, B. Heeren, ‘A systematic literature review of automated feedback generation for programming exercises,’ ACM Transactions on Computing Education 19(1), 2018. https://doi.org/10.1145/3231711.

[25] A. Luxton-Reilly et al., ‘Introductory Programming: A Systematic Literature Review,’ Proceedings Companion of ITiCSE 2018, 2018. https://doi.org/10.1145/3293881.3295779.

[26] I. Araya et al., ‘Predicting student performance in computing courses based on programming behavior,’ Computer Applications in Engineering Education, 2022. https://doi.org/10.1002/cae.22519.

[27] U. Omer, R. Tehseen, M. S. Farooq, A. Abid, ‘Learning analytics in programming courses: Review and implications,’ Education and Information Technologies, 2023. https://doi.org/10.1007/s10639-023-11611-0.

[28] J.-H. Yu, X.-Z. Chang, W. Liu, Z. Huan, ‘An online integrated programming platform to acquire students' behavior data for immediate feedback teaching,’ Computer Applications in Engineering Education 31(3), 520-536, 2023. https://doi.org/10.1002/cae.22596.

[29] W.-Y. Chang, ‘The Effects of Online Judge System on Motivation and Thinking in Programming Education: Structural Relationships between Factors,’ Journal of Korean Association of Computer Education 24(5), 1-16, 2021. https://doi.org/10.32431/kace.2021.24.5.001.

[30] G. Rodrigues, A. F. Monteiro, A. Osório, ‘Introductory Programming in Higher Education: A Systematic Literature Review,’ Third International Computer Programming Education Conference (ICPEC 2022), OASIcs 102, 4:1-4:17, 2022. https://doi.org/10.4230/OASIcs.ICPEC.2022.4.

APPENDIX A. REPOSITORY TRACE

• API controllers: backend/src/main/java/com/kodbtw/controller/.

• Adapters: backend/src/main/java/com/kodbtw/adapter/.

• Snapshot entities/repositories: backend/src/main/java/com/kodbtw/entity/ and repository/.

• Migrations: backend/src/main/resources/db/migration/V1 through V7.

• Frontend routes: frontend/src/App.tsx.

• SRS: documentation/requirements/SRS.md.

• Architecture: documentation/MODULE_ARCHITECTURE.md.

• Evaluation limitations and attempted commands: documentation/TESTING_AND_VALIDATION.md.